Digital Logic Circuit Design

VHDL Assignment-3
Latch & Flip-Flop 설계 보고서

보고서 요약 RS Latch, D Latch, D Flip-Flop, JK Flip-Flop, 비동기 Set/Clear D Flip-Flop을 각각 ① Logisim 게이트 수준 회로와 ② VHDL 코드로 구현하였다. 두 구현에 같은 입력 순서를 적용하여 동작이 일치함을 확인하였다.

과목	Digital Logic Circuit Design
이름	김우준
학번	2023092479
작성일	2026년 11월 7일

검증 요약: 5개 회로 모두 Logisim 게이트 회로와 VHDL 시뮬레이션 결과가 일치

1. 과제 개요

강의자료(VHDL_2)의 SR Latch, D Latch, D Flip-Flop, JK Flip-Flop 내용을 바탕으로 5개의 순차 논리 회로를 구현하였다. 각 문제는 Logisim에서 개별 논리 게이트만으로 만든 회로(래치·플립플롭 내장 부품 사용 안 함)와, 같은 기능의 VHDL 코드로 구성된다.

1.1 구현 항목

문제	회로	Logisim 게이트 구성	VHDL 핵심
1	RS Latch	NOR 2개 교차 연결	NOR 두 식의 되먹임
2	D Latch (enable)	NAND 4개 + NOT 1개	`IF en='1' THEN q <= d`
3	D Flip-Flop (상승 에지)	D Latch 2개(Master-Slave) + 클럭 인버터	`rising_edge(clk)`
4	JK Flip-Flop (상승 에지)	NAND 11개 + NOT 4개, Q/NQ 되먹임	Hold / Set / Reset / Toggle
5	D Flip-Flop + 비동기 Set/Clear	3입력 NAND 래치 + Clear 우선 논리	`IF clr ... ELSIF set ... ELSIF rising_edge`

1.2 개발 및 검증 환경

항목	환경
게이트 수준 회로	Logisim-evolution 5.0.0 (NOT / NAND / NOR 게이트, 입력·출력 핀)
VHDL 합성	GHDL 6.0.0 (`--synth`) → Yosys 0.70 (`proc`, RTL 셀 수준) → Graphviz 16.1 회로도
VHDL 시뮬레이션	Questa Altera Starter FPGA Edition 2025.2 (VHDL-2008, 자가검증 테스트벤치)

1.3 검증 방법

- **VHDL**: 문제마다 테스트벤치를 작성해 Questa에서 실행하고, 각 단계의 q·nq를 `assert`로 기대값과 비교하였다. 파형은 Questa Wave 창을 그대로 실었다.
- **Logisim**: 게이트 회로의 입력 핀을 VHDL 테스트벤치와 **같은 순서**로 바꾸며 Q·NQ를 확인하였다(5개 회로, 총 87개 확인 항목). 본문 그림은 해당 입력을 넣은 상태의 Logisim 회로이다(밝은 초록 = 1, 진한 초록 = 0).
- **합성**: GHDL로 각 VHDL을 합성해 넷리스트를 만들고, Yosys로 RTL 셀(`$dlatch`, `$dff`, `$aldff`, 논리 게이트, `$mux`) 수준까지 정리한 뒤 회로도로 그렸다. 회로도에서 팔각형은 입출력 포트, 마름모는 내부 신호, 타원은 상수이다.

2. 문제 1 - RS Latch (NOR)

두 NOR 게이트의 출력을 서로의 입력으로 되먹임한다. Q = NOR(R, NQ), NQ = NOR(S, Q)이다. S=1이면 Q가 1로 세트되고, R=1이면 0으로 리셋되며, S=R=0이면 이전 상태를 유지한다. S=R=1은 금지 입력으로, 두 출력이 모두 0이 되어 Q와 NQ가 서로 보수가 아니게 된다.

S	R	Q	NQ	동작
0	0	Q(t)	NQ(t)	Hold
1	0	1	0	Set
0	1	0	1	Reset
1	1	0	0	금지 입력

2.1 Logisim 게이트 회로

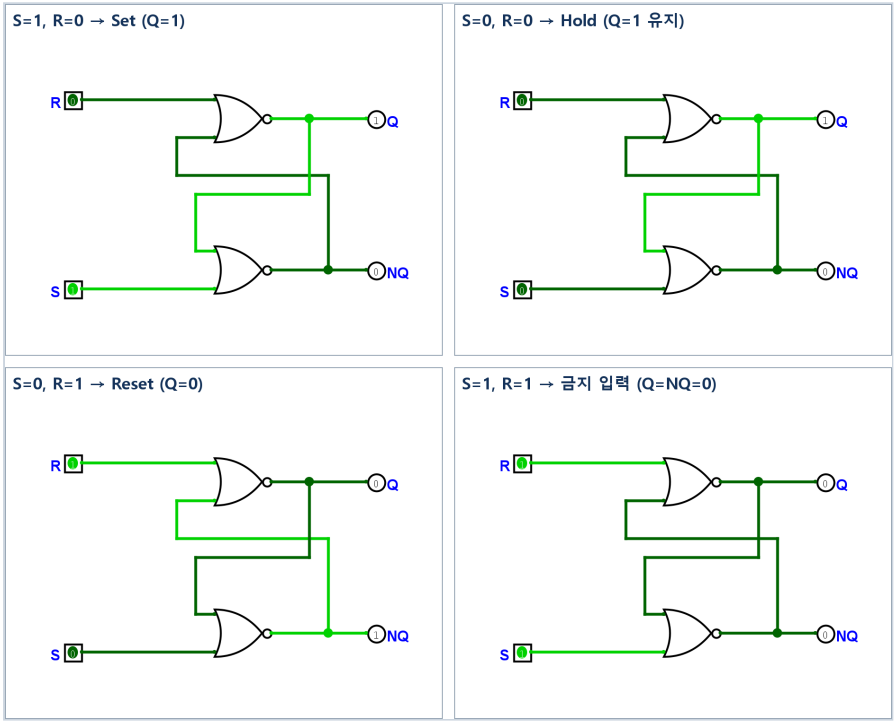


그림 1. NOR RS Latch의 Set / Hold / Reset / 금지 입력 동작

2.2 VHDL 코드 (rs_latch.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- RS latch made of two cross-coupled NOR gates
--   s r | q        nq
--   0 0 | hold     hold
--   1 0 | 1        0      (set)
--   0 1 | 0        1      (reset)
--   1 1 | 0        0      (forbidden)
ENTITY rs_latch IS
  PORT (
    s  : IN  std_logic;   -- Set input
    r  : IN  std_logic;   -- Reset input
    q  : OUT std_logic;   -- Output
    nq : OUT std_logic    -- Complement output
  );
END rs_latch;

ARCHITECTURE bhv OF rs_latch IS
  SIGNAL q_internal  : std_logic := '0';   -- Internal
signal to represent output q
  SIGNAL nq_internal : std_logic := '1';   -- Internal
signal to represent output nq
BEGIN
  q_internal  <= r NOR nq_internal;   -- NOR gate 1
  nq_internal <= s NOR q_internal;    -- NOR gate 2

  q  <= q_internal;
  nq <= nq_internal;
END bhv;
```

2.3 합성 회로도 (GHDL + Yosys)

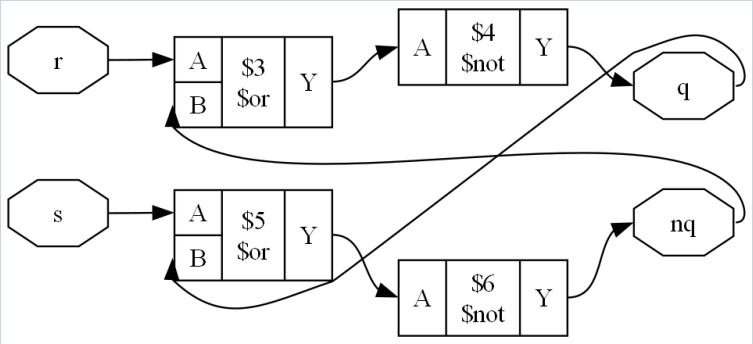


그림 2. rs_latch 합성 결과 - NOR(= $or + $not) 두 개의 출력이 서로의 입력으로 되먹임되어 게이트 회로와 같은 구조

2.4 시뮬레이션 결과 (Questa)

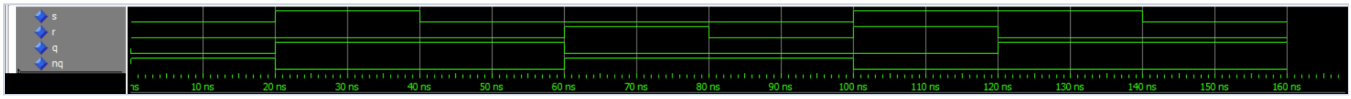


그림 3. tb_rs_latch 파형

구간 (ns)	0-20	20-40	40-60	60-80	80-100	100-120	120-140	140-160
S R	0 0	1 0	0 0	0 1	0 0	1 1	1 0	0 0
Q NQ	0 1	1 0	1 0	0 1	0 1	**0 0**	1 0	1 0
동작	Hold	Set	Hold	Reset	Hold	금지	Set	Hold

참고 금지 입력(S=R=1)에서 두 입력을 동시에 0으로 바꾸면 두 NOR 게이트가 같은 값으로 번갈아 바뀌는 발진이 생길 수 있다. 그래서 테스트에서는 R을 먼저 0으로 내려 Set 상태로 빠져나오게 하였다(120 ns).

PASS Questa 8단계 assertion error 0건, Logisim 9개 확인 항목 일치. Set / Reset / Hold / 금지 입력 모두 확인.

3. 문제 2 - D Latch (NAND)

강의자료의 D Latch 구성대로 입력 제어부(NAND 2개 + NOT)와 NAND SR 래치(NAND 2개)로 만들었다. EN=1이면 S'=NOT D, R'=D가 되어 Q가 D를 그대로 따라가고(transparent), EN=0이면 S'=R'=1이 되어 SR 래치가 이전 상태를 유지한다.

EN	D	Q(t+1)	동작
0	X	Q(t)	Hold
1	0	0	D를 따라감
1	1	1	D를 따라감

3.1 Logisim 게이트 회로

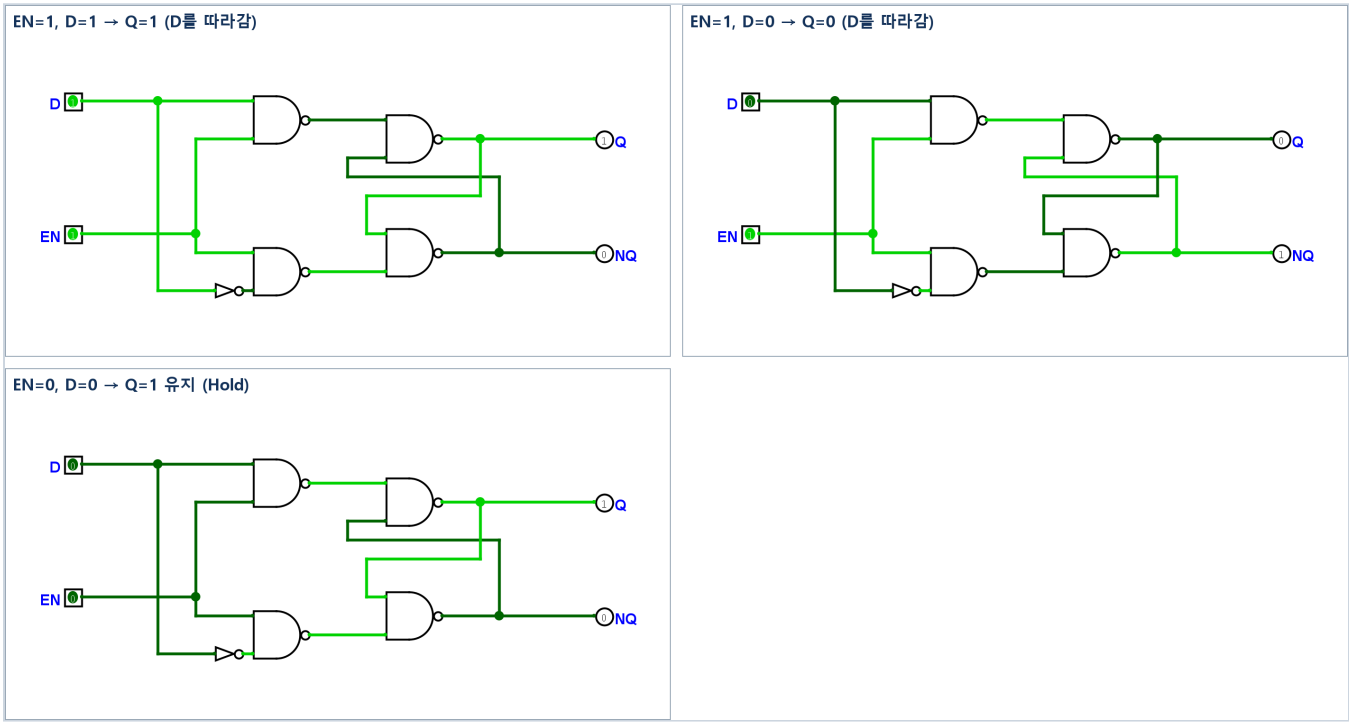


그림 4. NAND D Latch - EN=1에서 Q가 D를 따라가고, EN=0에서 이전 값(1)을 유지

3.2 VHDL 코드 (d_latch.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- D latch with enable: q follows d while en = '1',
holds while en = '0'
ENTITY d_latch IS
  PORT (
    en : IN  std_logic;    -- Enable signal
    d  : IN  std_logic;    -- Data input
    q  : OUT std_logic;    -- Output
    nq : OUT std_logic     -- Complement output
  );
END d_latch;
```

```
ARCHITECTURE bhv OF d_latch IS
  SIGNAL q_internal : std_logic := '0';   -- Internal
signal to represent output q
BEGIN
  q  <= q_internal;
  nq <= NOT q_internal;

  PROCESS(en, d)
  BEGIN
    IF (en = '1') THEN
      q_internal <= d;
    END IF;
  END PROCESS;
END bhv;
```

3.3 합성 회로도 (GHDL + Yosys)

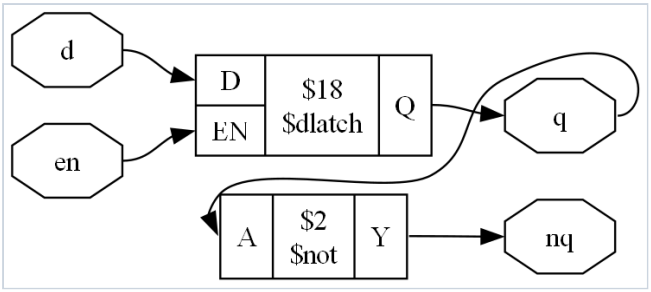


그림 5. d_latch 합성 결과 - EN으로 열리는 레벨 래치($dlatch)와 nq용 NOT

3.4 시뮬레이션 결과 (Questa)

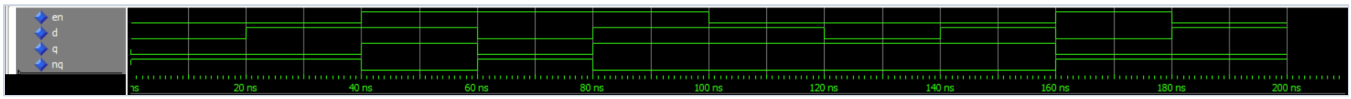


그림 6. tb_d_latch 파형

- 40~100 ns (en=1): q가 d의 변화(1 → 0 → 1)를 그대로 따라간다.
- 100~160 ns (en=0): d가 1 → 0 → 1로 바뀌어도 q는 1을 유지한다.
- 160~180 ns에 en=1, d=0이 되어 q=0이 되고, 180 ns 이후 en=0에서는 d=1이 되어도 q=0을 유지한다.

PASS Questa 10단계 assertion error 0건, Logisim 10개 확인 항목 일치.

4. 문제 3 - D Flip-Flop (Master-Slave)

강의자료의 Master-Slave 구조대로 문제 2의 D Latch 두 개와 클럭 인버터로 구성하였다. Master는 NOT CLK로, Slave는 CLK로 enable된다. CLK=0인 동안 Master가 D를 받아 두고, CLK가 0→1로 바뀌는 순간 Master가 닫히고 Slave가 열려 Master의 값이 Q로 전달된다. CLK=1인 동안에는 Master가 닫혀 있으므로 D가 바뀌어도 Q는 변하지 않는다. 따라서 Q는 상승 에지에서만 바뀐다.

4.1 Logisim 게이트 회로

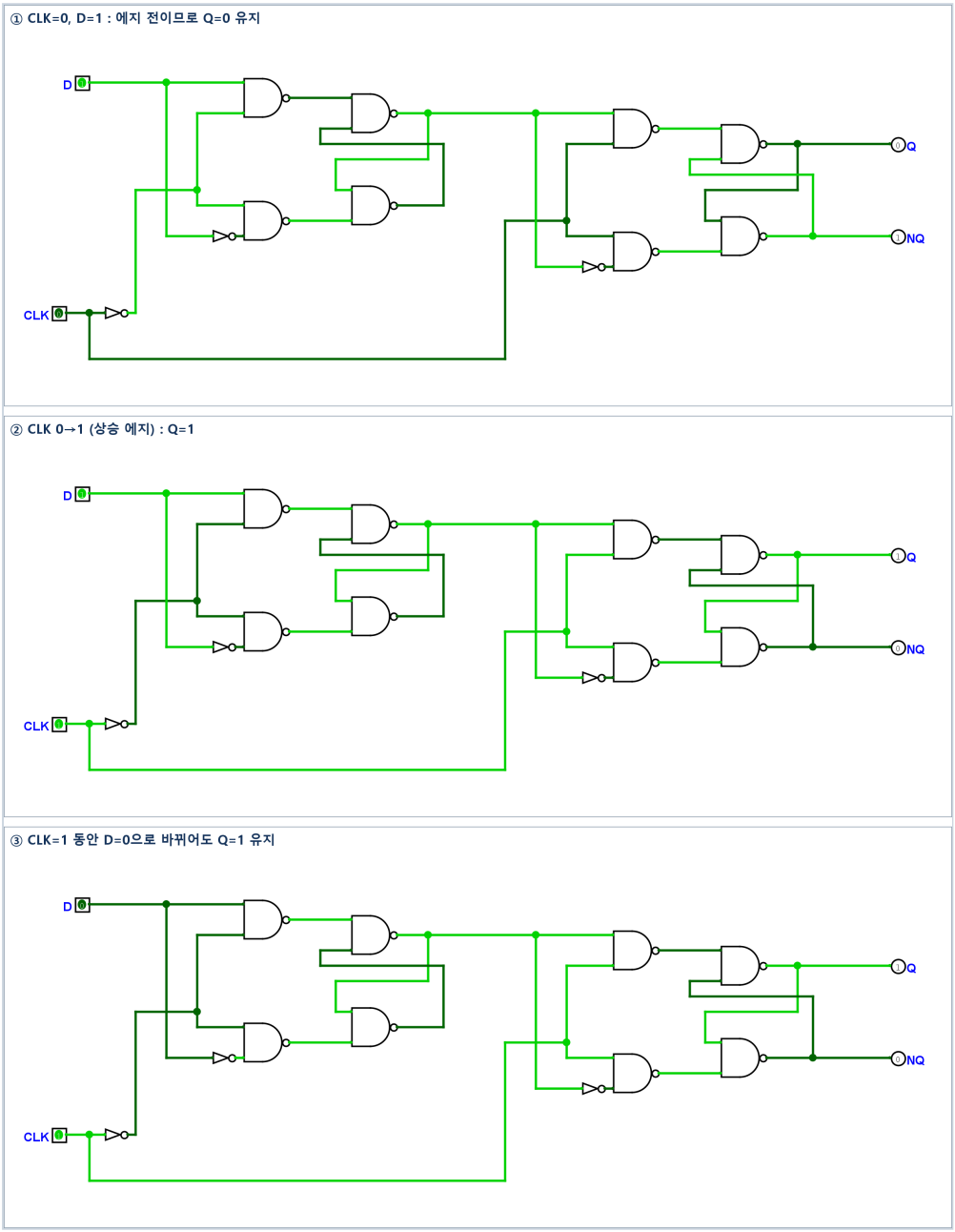


그림 7. Master-Slave D Flip-Flop - 상승 에지에서만 Q가 갱신됨

4.2 VHDL 코드 (d_flipflop.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Positive-edge-triggered D flip-flop: q takes d only
on the rising edge of clk
ENTITY d_flipflop IS
  PORT (
    clk : IN  std_logic;   -- Clock signal
    d   : IN  std_logic;   -- Data input
    q   : OUT std_logic;   -- Output
    nq  : OUT std_logic    -- Complement output
  );
END d_flipflop;
```

```
ARCHITECTURE bhv OF d_flipflop IS
  SIGNAL q_internal : std_logic := '0';   -- Internal
signal to represent output q
BEGIN
  q  <= q_internal;
  nq <= NOT q_internal;

  PROCESS(clk)
  BEGIN
    IF rising_edge(clk) THEN
      q_internal <= d;
    END IF;
  END PROCESS;
END bhv;
```

4.3 합성 회로도 (GHDL + Yosys)

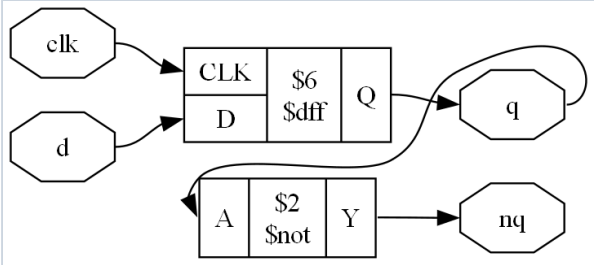


그림 8. d_flipflop 합성 결과 - 상승 에지 플립플롭($dff)과 nq용 NOT

4.4 시뮬레이션 결과 (Questa)

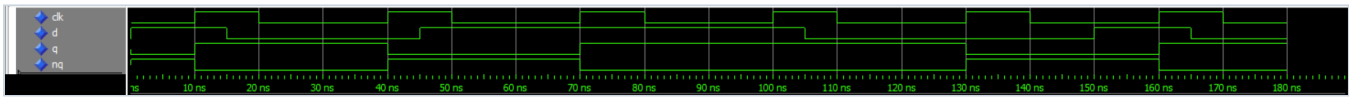


그림 9. tb_d_flipflop 파형

- 상승 에지는 10, 40, 70, 100, 130, 160 ns이고, 이때의 d(1, 0, 1, 1, 0, 1)가 q로 전달된다.
- 각 주기의 15 ns 지점(클럭이 1인 동안)에 d를 바꾸어도, 20 ns 지점의 하강 에지에서도 q는 변하지 않는다.

PASS Questa 6주기 × 4개 assertion error 0건, Logisim 25개 확인 항목 일치.

5. 문제 4 - JK Flip-Flop (NAND + 되먹임)

JK 플립플롭의 다음 상태식 **Q(t+1) = J·Q' + K'·Q**를 NAND 게이트로 만들고, 그 결과를 문제 3의 NAND Master-Slave D Flip-Flop 입력에 넣었다. 출력 Q와 NQ는 입력단 NAND로 되먹임된다. 입력단은 D = NAND(NAND(J, NQ), NAND(K', Q))이다.

J	K	Q(t+1)	동작
0	0	Q(t)	Hold
0	1	0	Reset
1	0	1	Set
1	1	Q'(t)	Toggle

설계 선택 강의자료 11쪽의 기본 NAND JK 회로는 CLK=1인 동안 J=K=1이면 출력이 계속 뒤집히는 race-around 현상이 있어 Logisim에서 발진한다. 과제의 요구(상승 에지 동작)와 VHDL 코드에 맞추기 위해, 되먹임 입력단을 에지 트리거 Master-Slave 구조 앞에 두어 상승 에지마다 한 번만 상태가 바뀌게 하였다.

5.1 Logisim 게이트 회로

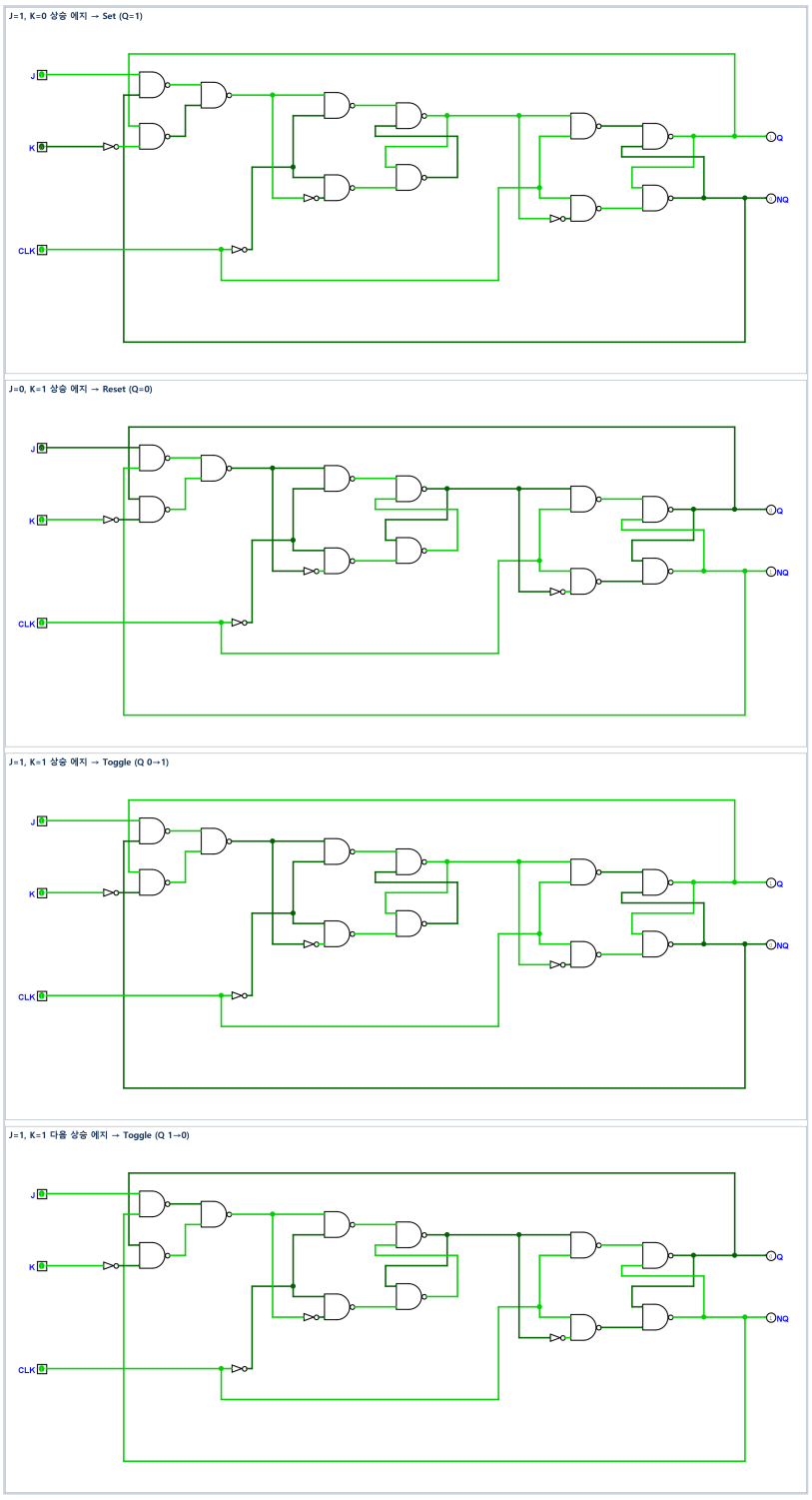


그림 10. JK Flip-Flop - Set / Reset / Toggle(2회) 동작

5.2 VHDL 코드 (jk_flipflop.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Positive-edge-triggered JK flip-flop
--   j k | q(t+1)
--   0 0 | q(t)       Hold
--   0 1 | 0          Reset
--   1 0 | 1          Set
--   1 1 | not q(t)   Toggle
ENTITY jk_flipflop IS
  PORT (
    clk : IN  std_logic;   -- Clock signal
    j   : IN  std_logic;   -- J input
    k   : IN  std_logic;   -- K input
    q   : OUT std_logic;   -- Output
    nq  : OUT std_logic    -- Complement output
  );
END jk_flipflop;

ARCHITECTURE bhv OF jk_flipflop IS
  SIGNAL q_internal : std_logic := '0';   -- Internal
signal to represent output q
BEGIN
  -- Drive the output ports with the internal signal
  q  <= q_internal;
  nq <= NOT q_internal;

  PROCESS(clk)
  BEGIN
    IF rising_edge(clk) THEN
      IF (j = '1' AND k = '0') THEN
        q_internal <= '1';                -- Set
      ELSIF (j = '0' AND k = '1') THEN
        q_internal <= '0';                -- Reset
      ELSIF (j = '1' AND k = '1') THEN
        q_internal <= NOT q_internal;     -- Toggle
      END IF;                             -- j = k =
'0' : Hold
    END IF;
  END PROCESS;
END bhv;
```

5.3 합성 회로도 (GHDL + Yosys)

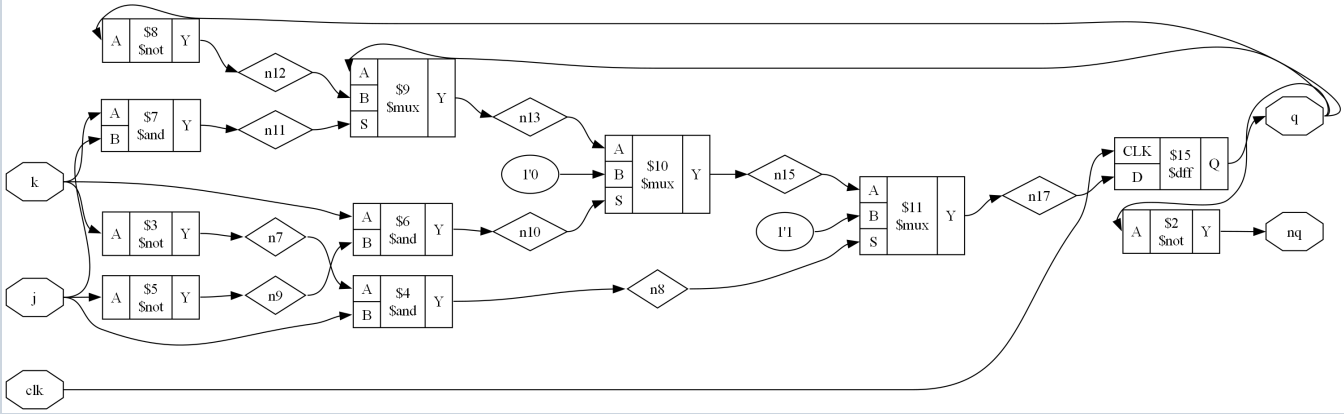


그림 11. jk_flipflop 합성 결과 - J·K 조건(AND/NOT)으로 Set·Reset·Toggle 값을 고르는 $mux 체인과 $dff, Q 되먹임

5.4 시뮬레이션 결과 (Questa)

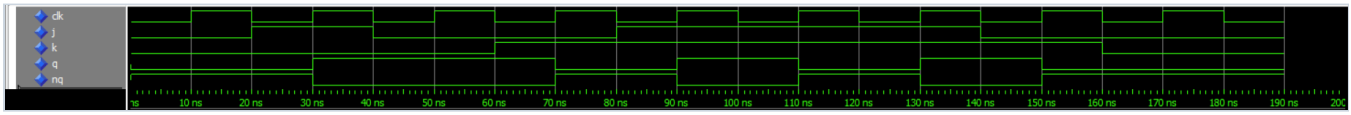


그림 12. tb_jk_flipflop 파형

상승 에지 (ns)	10	30	50	70	90	110	130	150	170
J K	0 0	1 0	0 0	0 1	1 1	1 1	1 1	0 1	0 0
Q	0	1	1	0	1	0	1	0	0
동작	Hold	Set	Hold	Reset	Toggle	Toggle	Toggle	Reset	Hold

PASS Questa 9주기 assertion error 0건, Logisim 28개 확인 항목 일치. 네 가지 동작 모드 모두 확인.

6. 문제 5 - 비동기 Set / Clear D Flip-Flop

문제 3의 Master-Slave D Flip-Flop에서 네 NAND 래치 게이트를 3입력 NAND로 바꾸고, 활성-0 제어 신호 nCLR, nPRE를 넣었다. nCLR = NOT clr, **nPRE = NAND(set, nCLR)**이므로 clr=1이면 nPRE가 강제로 1(비활성)이 되어 Clear가 Set보다 우선한다. 두 신호는 클럭 경로를 거치지 않고 래치 게이트에 직접 들어가므로 클럭과 관계없이 즉시 출력이 바뀐다.

clr	set	clk	Q	동작
1	X	X	0	비동기 Clear (최우선)
0	1	X	1	비동기 Set
0	0	↑	D	일반 D Flip-Flop
0	0	그 외	Q(t)	유지

6.1 Logisim 게이트 회로

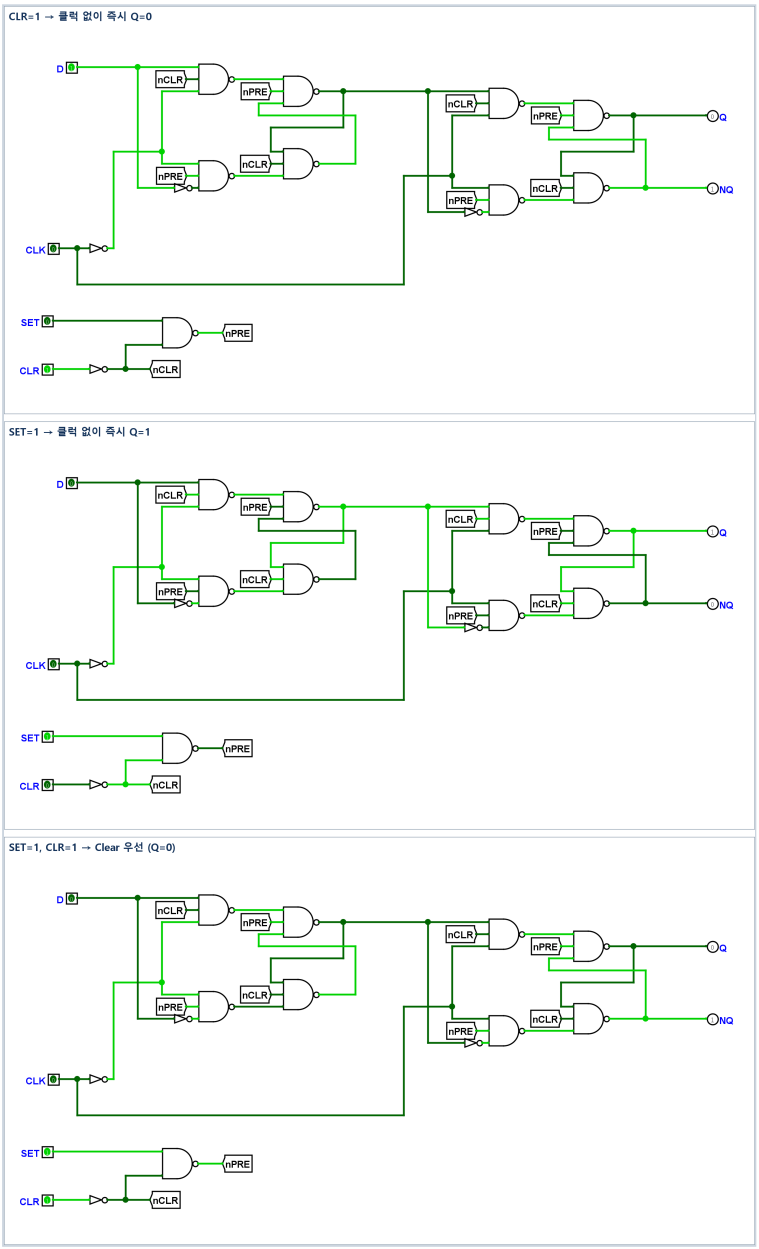


그림 13. CLR / SET이 클럭 없이 즉시 적용되고, 둘 다 1이면 Clear가 우선

6.2 VHDL 코드 (d_flipflop_sc.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Positive-edge-triggered D flip-flop with
asynchronous Set and Clear
--   clr = '1'           -> q = '0' immediately
(highest priority)
--   set = '1', clr = '0' -> q = '1' immediately
--   otherwise           -> normal D flip-flop
ENTITY d_flipflop_sc IS
  PORT (
    clk : IN  std_logic;   -- Clock signal
    d   : IN  std_logic;   -- Data input
    clr : IN  std_logic;   -- Asynchronous clear
signal
    set : IN  std_logic;   -- Asynchronous set signal
    q   : OUT std_logic;   -- Output
    nq  : OUT std_logic    -- Complement output
  );
END d_flipflop_sc;

ARCHITECTURE bhv OF d_flipflop_sc IS
  SIGNAL q_internal : std_logic := '0';   -- Internal
signal to represent output q
BEGIN
  q  <= q_internal;
  nq <= NOT q_internal;

  PROCESS(clk, clr, set)   -- Include clr and set for
asynchronous handling
  BEGIN
    -- Asynchronous clear has the highest priority
    IF clr = '1' THEN
      q_internal <= '0';
    -- Asynchronous set has the second-highest
priority
    ELSIF set = '1' THEN
      q_internal <= '1';
    -- Synchronous clock-based behavior
    ELSIF rising_edge(clk) THEN
      q_internal <= d;
    END IF;
  END PROCESS;
END bhv;
```

6.3 합성 회로도 (GHDL + Yosys)

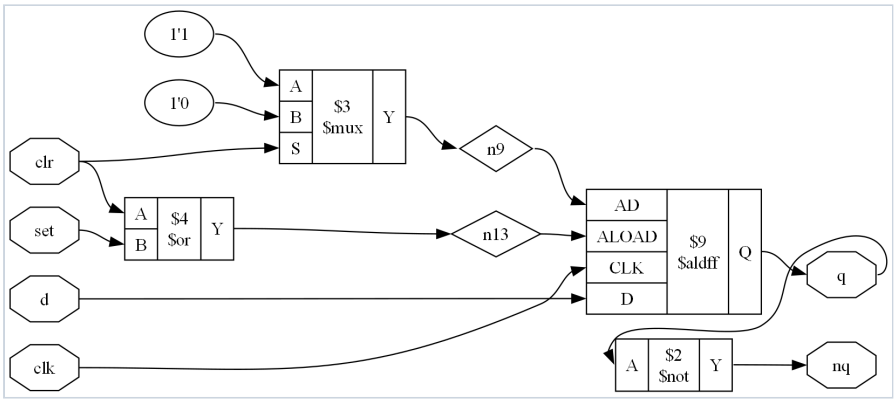


그림 14. d_flipflop_sc 합성 결과 - clr 또는 set이면 클럭과 무관하게 AD 값을 로드하는 $aldff. AD = clr ? 0 : 1 이므로 Clear가 우선

6.4 시뮬레이션 결과 (Questa)

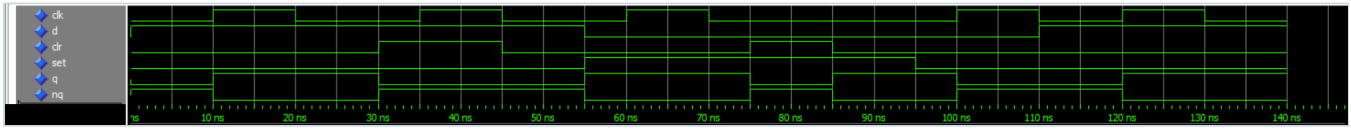


그림 15. tb_d_flipflop_sc 파형

시각 (ns)	10	30	35	55	60	75	85	100	120
사건	clk↑ d=1	clr=1	clk↑	set=1	clk↑ d=0	clr=1 (set=1)	clr=0	clk↑ d=0	clk↑ d=1
Q	1	0 (즉시)	0 유지	1 (즉시)	1 유지	0 (Clear 우선)	1 (Set)	0	1

PASS Questa 13개 assertion error 0건, Logisim 15개 확인 항목 일치. 비동기 Clear/Set, Clear 우선, 일반 D 동작 모두 확인.

7. 종합 검증 결과

문제	Logisim 게이트 수	Questa 테스트벤치	Logisim 확인 항목	결과
1 RS Latch	NOR 2	8단계, error 0	9개 일치	PASS
2 D Latch	NAND 4, NOT 1	10단계, error 0	10개 일치	PASS
3 D Flip-Flop	NAND 8, NOT 3	6주기, error 0	25개 일치	PASS
4 JK Flip-Flop	NAND 11, NOT 4	9주기, error 0	28개 일치	PASS
5 D FF + Set/Clear	NAND 9, NOT 4	13단계, error 0	15개 일치	PASS

PASS 5개 문제 모두 게이트 수준 구현(Logisim)과 VHDL 구현(Questa)에 같은 입력 순서를 넣었을 때 같은 출력이 나왔다.

8. 설계 시 고려사항

항목	내용
래치 초기 상태	게이트로 만든 래치는 전원 투입 직후 상태가 정해지지 않아 Logisim에서 Q/NQ가 E(빨간색)로 보인다. Reset(또는 clr) 입력을 한 번 넣으면 정상 값이 된다. VHDL에서는 내부 신호 초기값을 '0'으로 두었다.
NQ 출력	게이트 회로와 출력을 맞추기 위해 모든 VHDL 설계에 보수 출력 nq를 두었다. 강의자료 코드처럼 출력 포트를 읽지 않도록 내부 신호(q_internal)를 사용하였다.
에지 검출	강의자료는 `clk'event AND clk='1'`을 사용하지만, 과제 지시에 따라 같은 의미의 `rising_edge(clk)`를 사용하였다.
JK race-around	레벨 트리거 NAND JK의 발진을 피하기 위해 Master-Slave 에지 구조에 되먹임 입력단을 두었다(5장 참고).
Clear 우선순위	게이트 회로에서 set과 clr이 동시에 1이면 두 출력이 모두 1이 되므로, nPRE = NAND(set, NOT clr)로 Set을 막아 VHDL의 IF-ELSIF 우선순위와 같게 하였다.

9. 결론 및 제출 파일

다섯 회로 모두 과제 조건대로 Logisim 게이트 수준 회로와 VHDL 코드 두 가지로 구현하였고, 합성 회로도와 시뮬레이션 파형으로 동작을 확인하였다. 같은 입력 순서에서 두 구현의 출력이 모두 일치하므로 같은 논리 기능을 나타낸다.

폴더	VHDL 소스	Logisim 프로젝트
P1_rs_latch	rs_latch.vhd	rs_latch.circ
P2_d_latch	d_latch.vhd	d_latch.circ
P3_d_flipflop	d_flipflop.vhd	d_flipflop.circ
P4_jk_flipflop	jk_flipflop.vhd	jk_flipflop.circ
P5_d_flipflop_sc	d_flipflop_sc.vhd	d_flipflop_sc.circ
testbench	tb_*.vhd 5개	-